



Ink Morrow
WHERE STORIES GROW CLAWS

5.0 EDITION

Operations & Recovery Handbook

Operate the new game, recover safely and protect historical data

For the owner-operator / September 2026

Operations & Recovery Handbook

Operate a private, single-owner InkMorrow 5.0 installation. Protect data first, distinguish a failed purchase from a free operation, and recover without replaying work.

Contents

The installation you are operating

Install without touching old data

Resolve configuration and storage

First-run and access recovery

Provider roles and explicit purchases

Two different backup products

Restore and update safely

Network exposure is an operator decision

Incident triage and healthy completion

Optional consistency operations

Episode and relationship support

Fourth-wall support

Direction and memory support

01 The installation you are operating

InkMorrow serves its static browser interface and authenticated API from one Node process. The production entry point is `backend/server.js`; its default address is `127.0.0.1:3000`. It is a playable-fiction game, not the retired manuscript suite. Opening the interface, reading, local state changes and backup do not buy prose.

The live surfaces are Your stories, Visual Library, Start a story, the reader and Settings. The default player stays outside the cast. New narration is deliberate; there is no speculative next-page queue, automatic cast portrait backfill, audiobook queue or background memory extraction in the 5.0 runtime.

Use one process per data directory. SQLite state, normalized images and encrypted provider-vault records form one installation. Downloaded `.inkmorrow5` saves are portable story copies, not full installation backups.

For this project's development, the authoritative checkout is `/home/rthorman/src/ink-morrow-5` under WSL. Windows tools may operate on that tree; do not maintain an independent Windows source copy. Production deployment and starting a server remain explicit operator actions, separate from merging code.

02 Install without touching old data

Use a supported Node 22 runtime providing node:sqlite and a compatible sharp binary; the package declares Node 22.5 or newer. Verify the actual installed runtime, especially when mixing WSL and Windows tools. The setup script checks prerequisites and prints actionable errors instead of starting a partial server.

From the repository root, run `bash setup.sh`. It installs production dependencies, creates missing configuration and prepares the new data location. An existing dependency tree is updated in place; `--dev` includes developer packages. Only explicit `--clean` authorises replacing the listed `node_modules` directories. Do not use `clean setup` as a casual response to a runtime error.

Review `backend/.env` before the first launch. Supply a provider key only if you want generated play; a curated opening and existing local records can be read without one. Placeholder keys cannot generate. A provider rejection (401) means repair its credential, not the owner login. Use a separate profile for a UI-entered key; the environment profile is read-only. Do not copy old storage overrides.

The default new database is `database-v5/ink-morrow-5.db` relative to the repository. A recognized 4.x database is refused without adoption. Keep older installations and data separate; there is no supported in-place upgrade from that series.

When you are ready to run, `npm start` from the repository launches the backend. Keep the printed one-time setup code private and use it in the browser to set the owner password. A successful launch is not permission to expose the port to the internet.

03 Resolve configuration and storage

backend/.env is loaded by the executable and password-reset command. Existing process environment values take precedence. Both resolve relative DATA_DIR and DB_PATH values from backend/, not from the shell's current directory.

Configuration	Database	Media root
Neither override	database-v5/ink-morrow-5.db	database-v5
DATA_DIR only	DATA_DIR/ink-morrow-5.db	DATA_DIR
DB_PATH only	That exact file	Its containing directory
Both	That exact file	DATA_DIR
DB_PATH=:memory:, no DATA_DIR	In-memory database	Unique disposable temporary directory

DATA_DIR must be a directory, never :memory:. In-memory mode is for isolated tests and deliberate ephemeral use; it is not persistent storage. An explicit DATA_DIR with an in-memory database does not make its story records durable.

PORT defaults to 3000 and HOST to 127.0.0.1. OPENROUTER_API_KEY supplies the built-in read-only credential; Settings stores explicit role assignments and optional vault credentials. AI_TIMEOUT_MS and IMAGE_TIMEOUT_MS bound individual calls, not a guaranteed end-to-end completion time.

CONTINUITY_MODEL and old authoring/retry settings do not revive the retired runtime. Memory support is assigned through Settings. Quality Off makes no separate memory request; enabling quality is a story-path choice.

04 First-run and access recovery

First-run setup requires the terminal code and a 15–128-character password. Use a distinctive passphrase. Browser sessions use an `HttpOnly`, `SameSite=Strict` cookie; state-changing requests also need the session CSRF token and same-origin request. The executable remains sealed even with `NODE_ENV=test`.

Lock revokes the current browser session. Password change revokes other sessions. After a process restart, a remembered browser login may still be valid while vault credentials remain locked; re-enter the owner password to unlock them. Do not overwrite a provider profile merely to solve a locked vault.

If the password is lost, stop the exact installation, confirm its `DATA_DIR` and `DB_PATH`, and keep a cold backup. From backend, run `npm run auth:reset -- --yes`. This removes the owner, browser sessions and saved provider credentials from the selected database. Stories and media remain. The next start prints a new setup code.

The reset command refuses a missing database or `:memory:` and never creates a blank recovery database. An old family is rejected before any reset. If it says nothing was removed, inspect the path rather than deleting files or guessing.

Terminal access is powerful: someone able to read or replace the data files is outside the browser-password security boundary. Use operating-system protections and encrypted storage where appropriate.

05 Provider roles and explicit purchases

Settings separates the storyteller (internal Scribe role), memory support (Archivist role) and Illustrator. A role names one provider profile and model. Saving a choice is local; browsing its model catalogue contacts that provider but does not purchase story generation.

The built-in OpenRouter profile reads environment credentials. User-supplied profiles can use process-session credentials or the encrypted vault. An unavailable role is not silently replaced. Do not place secrets in story fields or assume an arbitrary OpenAI-compatible endpoint supports every operation.

Standard play purchases one text response. Quality can add bounded checks using standard, memory or both roles. Painting is a separate one-attempt operation. There is no auto-retry after uncertain transport failure, automatic successor, or background recap generation.

Before authorising a request, read its role/model, data exposure and call ceiling. An estimate is not a price cap; configure an upstream spending limit where supported. Failed or rejected work can still have known or unknown charges. Changing providers does not refund a previous attempt.

Use a fresh explicit action only after free state reconciliation. Do not replay a POST with a new idempotency key merely because the browser waited too long.

06 Two different backup products

Download a playable save for each important story. It contains all paths, cast, private facts, settings, illustrations and aggregate spend. It excludes credentials, provider configuration, consent and pending request identities. Import always creates a new story. Keep saves private and unencrypted-file risks visible.

Story saves also include frozen world/Scribe/character references, covers and portraits, but not the reusable catalogues or their separate AI journals. Use the full installation backup for catalogue entries and reference-development or image spend. Deleting an entry scrubs its reusable content and image, retains its spend record, and leaves every story copy alone. This catalogue deletion has no in-app undo.

A cold installation backup preserves the complete configured database and media roots, including auth and encrypted vault data. Stop the process and confirm it has exited before copying. If database and media use different roots, capture both. Protect a separate copy of necessary configuration; provider keys in .env are secrets and should not enter ordinary issue attachments or shared archives.

Copy the database with any associated -wal, -shm and -journal files still present after shutdown. Never discard a WAL because the main database looks complete. Keep the directory relationship intact. A live ordinary filesystem copy is not a transactionally coherent SQLite backup.

Name and date backups, record the app version, and keep a copy off the same device when device-loss recovery matters. Test restoration to a separate location. A synced folder or downloaded EPUB alone is not proof of recovery.

The application's private scratch-copy database inspection protects older source files during startup preflight. It is not a backup service and not a substitute for stopping writers before an operator copy.

07 Restore and update safely

Schema 23 adds durable AI reference-development records without rewriting schema-22 catalogues, story history or earlier migration checksums. This is a supported 5.0 migration, not 4.x adoption. Keep a matched cold backup before updating; older code cannot read the new schema.

For a playable save, use Import a playable save, check the preview, then Import as a new story. Verify an old moment, another path and an illustration. Configure provider roles separately and review any future purchase. Import never starts an interrupted operation.

For an installation restore, stop the target process and retain its current data in a separate recoverable location. Restore the complete cold backup into a fresh destination, including media and any retained database sidecars. Point the matching application version at that destination. Do not overlay files from two backups or replace only the database while leaving mismatched media.

Check the family/version acceptance, unlock, inspect several stories and paths, and export a reading copy before declaring success. A refused database is a reason to investigate its provenance, not to edit the family marker manually.

Before updating within 5.0, record the exact code revision, stop the process, take a cold backup, and read release notes. Test on an isolated copy where practical. Migration checksums and SQLite integrity checks must pass; failure does not authorise ledger surgery.

Rolling back code alone is not a safe schema rollback. Use a matched prior code-and-data backup in a separate destination if a later version cannot be read. The 4.x line is not a rollback target for a newly created 5.0 database.

08 Network exposure is an operator decision

Keep the default loopback bind for local use. WSL and Windows networking details can differ; verify the actual listening address rather than assuming a terminal message proves remote reachability or firewall isolation.

For remote access, prefer an HTTPS reverse proxy on loopback with an explicit `ALLOWED_HOSTS` list and `TRUST_PROXY=1`. Trust is limited to the loopback proxy, not arbitrary client-supplied forwarding headers. Configure the proxy and its certificate deliberately, then verify Secure cookies and rejected unapproved Host headers.

Direct non-loopback HTTP requires `ALLOW_INSECURE_LAN=1`. This is a warning-bearing opt-in, not encryption. Passwords, sessions, story content and provider traffic to the local application would cross that connection in clear text. Do not expose it directly to the public internet.

This is one local owner's installation, not a hosted multi-user account system. The access password does not encrypt the database, media or downloaded files. Do not treat Living-world resistance as a security control; fictional authority does not govern real API access.

The Security handbook describes the threat boundaries. Changing exposure, proxy trust or credential handling is a security change requiring verification, not an incidental convenience setting.

09 Incident triage and healthy completion

First identify whether the problem is access, storage, provider configuration, network transport, invalid model output or stale story state. Record the visible error code and request time without exporting private prose or credentials.

Symptom	Safe next check
Database refused	Verify family, path, version and stopped writers
Vault unavailable after restart	Re-enter the owner password
Missing model role	Inspect its saved provider/model assignment
Reply failed or timed out	Refresh freely; inspect known/unknown call costs
Repeated approach unchanged	Read the existing ruling; no purchase is needed
Image/book unavailable	Check the exact retained media files and storage
Fact seems missing	Check path, visibility, retirement and older recall
Another tab changed the story	Refresh before another deliberate mutation

Do not run the old continuity repair pipeline, restore manual APIs, or remove history to solve a 5.0 memory complaint. The working set is bounded, but immutable fact history remains available for retrieval. Corrections are explicit local records, not paid background extraction.

A healthy maintenance result includes an accepted 5.0 database, successful unlock, local reading, correct path state, intact illustrations and a verified save. Test a paid operation only with explicit spending authority; deterministic fixtures are sufficient for regression checks.

Keep the prior backup until the owner has verified the result. Report exactly what changed, what was tested and what remains uncertain. A successful code merge does not mean the running installation was deployed or restarted.

10 Optional consistency operations

Quality is off by default. Story preferences select Standard, Memory or Both; Settings assigns the standard storyteller and memory-support provider/model independently. Saving a role or changing a preference makes no model call. All selected roles must resolve before a paid operation begins, and are rechecked between calls and before commit. A changed role, missing credential, locked vault or changed story stops work rather than silently downgrading quality.

Off permits one call. A single reviewer role permits four total calls; Both permits six. A first-pass acceptance needs two or three respectively. The sole repair is followed by all selected reviews. Invalid reviewer output or transport failure stops immediately, with no automatic retry. Review shows the role models, bounded private context exposure and cost/latency tradeoff; previous standard-play consent is not authority for quality purchases.

Inspect story totals and Recent model calls & costs after failure. Each dispatched call is journalled before transport. Known earlier charges and later unknown attempts remain separate; do not interpret a failed scene as free. Restart marks pending calls and their parent request interrupted. A late return may improve billing knowledge but cannot save the abandoned draft. Use a new explicit action only after reviewing the error; never replay paid POSTs to recover a lost response. No rejected draft or reviewer explanation is persisted as story history.

11 Episode and relationship support

Catch me up is a local read, not an AI repair pipeline. It returns current-path public reminders and evidence links. A temporarily unreachable server can prevent the recap from loading, but opening it never buys narration. Private motives and other paths are excluded. Return after an absence does not advance the world.

Relationship descriptions distinguish caring, trust, cooperation and expectations. Check recorded evidence before correcting an apparent disagreement. Corrections do not rewrite earlier prose. Episode phases describe development, payoff and aftermath; only the player ends an episode. Ending early is valid, and a retired goal is not treated as an automatically completed one.

Rewind and playable saves preserve these fields and remap payoff evidence into the copied story. Failed path and episode saves retain their dialog text. Refresh after a conflicting change; no automatic paid retry or alternate-path creation is attempted. Optional consistency-model calls follow the reviewed quality pipeline described above; local path and episode actions never start that pipeline.

12 Fourth-wall support

In Living-world Story preferences, Characters may break the fourth wall offers Never, Rarely and Freely. Never is the default. Rarely is permission for at most one structured address in six narrated passages, not a promise to produce an address on schedule. It stays inactive in Story-shaping and for outside-story Ask. Changing the setting costs nothing and does not reset its cooldown.

The preference and cooldown restore through reload, rewind and playable saves. The character's named address remains in its passage and book exports. If a provider returns a forbidden address, the response fails without saving partial text; its charge can still count. Refresh does not retry it. Ordinary model prose can still violate instructions; inspect the selected mode and storyteller, and never represent this control as proof that a model will preserve immersion.

13 Direction and memory support

If a story keeps returning to a topic, inspect its visible ongoing focus. A new Steer defaults to this moment; Keep this focus must be selected explicitly. Clear ongoing focus is local and does not alter previous prose. Style changes affect future play and do not revoke an already granted structured outcome.

Recall older facts searches public history without a model. Use distinctive words; results are bounded to 32. A missing fact may be retired, secret, or on another path, not lost through compaction. Source links distinguish local corrections from narrated evidence. Never retire a fact merely to keep the game running. Shelf pagination exposes older stories through More stories and Previous stories.